

INTRODUCCION

El juego tiene como objetivo desarrollarlo en Java llamado Super Elizabeth Sis, controlar a una princesa llamada Elizabeth a través de un escenario por plataformas flotantes, enemigos que vuelan e ítems que ayudan a la princesa a llegar al castillo al final del mapa. Utilizamos para desarrollarlo conceptos vistos en la materia (Programación 1), aplicada a objetos, arreglos y detección de colisiones. Lo realizamos en un equipo de tres personas facilitando el desarrollo y poder integrarlo finalmente al proyecto.

DESCRIPCION

- Clase Juego:

Controlar la lógica del juego, actualizar los objetos y gestionar las variables principales como personaje, enemigos, ítems, juegoGanado o juegoTerminado.

Se separó parte de la información de esta misma para usarlas en la clase Mapa y Hud para mejorar el diseño de código.

PROBLEMAS ENCONTRADOS:

El personaje atravesaba plataformas lateralmente, el mapa generaba plataformas imposibles de llegar, los enemigos atravesaban las islas flotantes.

SOLUCIONES:

Se agregaron verificaciones para las colisiones laterales, se ajustó la generación de plataformas, tanto su "Y" como su "X" para hacerlas aparecer con aleatoriedad, pero siendo posible para el personaje llegar a subir. Implementamos que los enemigos al chocarse contra una isla suban para arriba haciendo que no la atravesaran si no "esquivarlas".

- Clase Personaje:

Su movimiento, salto, gravedad y disparo, aprender sus variables importantes como x, y, vidas, disparo, disparoPlus o disparoTriples. Administrar el personaje el cual es el que le da vida al juego.

PROBLEMAS ENCONTRADOS:

El personaje atravesaba plataformas desde abajo o los costados, y el disparoPlus podía dispararse múltiples veces simultáneamente, disparaba y el contador seguía estando en 1 o 2.

SOLUCIONES:

Se ajustaron las plataformas para que las islas flotantes no coincidan con la altura del personaje y no aparezcan a la mitad de su cuerpo. Se incorporo a la colisión agregando controles que corrigen su posición cuando entra en contacto con una plataforma. El disparoPlus no descontaba al usarlo, se soluciono permitiendo un solo disparo especial activo a la vez y descontándolo al momento de utilizarlo.

- **Clase Mapa:**

Genera plataformas e islas flotantes, dibuja el escenario.

Tomamos la decisión de crear esta clase para reducir la cantidad de responsabilidades de la clase Juego.

- **Clase HUD:**

Mostrar las vidas, contadores de disparos, mensaje de victoria o derrota.

Separamos la interfaz visual de la clase Juego por la misma razón anterior que tomamos para la clase Mapa.

IMPLEMENTACION

Se presentan algunos métodos más importantes para el proyecto. Estos corresponden a funcionalidades centrales del juego como la generación del mapa, movimiento del personaje, detección de colisiones o los proyectiles de disparo. El código completo se encuentra incluido en el repositorio entregado junto con el trabajo practico.

- CREAR MAPA:

```
- private void crearMapa() {  
-     int indice = 0;  
-  
-     plataformas[indice] = new Plataforma(400,  
580, 620, 40);  
-     indice++;  
-  
-     for (int x = 980; x <= 5900 && indice <  
plataformas.length; x += 420) {  
-         plataformas[indice] = new  
Plataforma(x, 580, 340, 40);  
-         indice++;  
-     }  
-  
-     plataformas[indice] = new  
Plataforma(6100, 580, 420, 40);  
-     indice++;  
-  
-     plataformas[indice] = new  
Plataforma(6500, 580, 600, 40);  
-     indice++;  
-  
-     double xIsla = 650;  
-     int alturaAnterior = 490;  
-  
-     while (indice < plataformas.length &&  
xIsla < 6100) {  
-  
-         xIsla += 230 + Math.random() * 260;  
-  
-         int nivel = (int) (Math.random() *  
4);  
-  
-         int y;  
-  
-         if (nivel == 0) {  
-             y = 485;  
-         } else if (nivel == 1) {  
-             y = 440;  
-         } else if (nivel == 2) {  
-             y = 410;
```

```

-         } else {
-             y = 390;
-         }
-
-         if (Math.abs(y - alturaAnterior) >
80) {
-             if (y < alturaAnterior) {
-                 y = alturaAnterior - 60;
-             } else {
-                 y = alturaAnterior + 60;
-             }
-         }
-
-         double ancho = 120 + Math.random() *
110;
-
-         plataformas[indice] = new
Plataforma(xIsla, y, ancho, 25);
-         indice++;
-
-         alturaAnterior = y;
-
-         if (Math.random() < 0.25) {
-             xIsla += 120 + Math.random() *
120;
-         }
-     }
- }

```

Este método se encarga de generar el escenario para el juego, las plataformas del suelo o las flotantes, se incorpora aleatoriedad en la ubicación de las islas para que cada mapa o juego sea diferente al anterior.

- SALTAR Y APLICAR GRAVEDAD:

```

public void saltar() {
    if (!estaEnElAire) {
        velocidadY = FUERZA_SALTO;
        estaEnElAire = true;
    }
}

```

```

    }
}

public void aplicarGravedad() {
    yAnterior = y;

    if (estaEnElAire) {
        velocidadY += GRAVEDAD;
        y += velocidadY;
    }
}

```

Este método saltar() permite que la princesa solo salte cuando se encuentra apoyada sobre una plataforma. Se ejecuta, asigna una velocidad vertical negativa que hace que el personaje vaya para arriba y se actualiza el estado para saber si esta en el aire.

El método aplicarGravedad() simula la gravedad en el juego. Si el personaje esta en el aire, aumenta su velocidad vertical modificando donde esta situado, genera el efecto caída haciéndolo más realista.

- Choca la cabeza con plataforma o de costado.
- **public boolean** chocaCabezaConPlataforma(Plataforma plataforma) {
 - **boolean** cruzaLaParteInferior =
 - **yAnterior** - **alto** / 2 >=
 - plataforma.bordeInferior()
 - **&&** bordeSuperior() <=
 - plataforma.bordeInferior();
 -
 - **boolean** estaDentroDelAncho =
 - **bordeDerecho()** >
 - plataforma.bordeIzquierdo()

```

-         && bordeIzquierdo() <
plataforma.bordeDerecho());
-
-         return velocidadY < 0 &&
cruzaLaParteInferior && estaDentroDelAncho;
-     }
-
-     public boolean
chocaCostadoConPlataforma(Plataforma plataforma) {
-
-         boolean seSuperponeEnX =
bordeDerecho() >
plataforma.bordeIzquierdo()
-         && bordeIzquierdo() <
plataforma.bordeDerecho();
-
-         boolean seSuperponeEnY =
bordeInferior() >
plataforma.bordeSuperior()
-         && bordeSuperior() <
plataforma.bordeInferior();
-
-         boolean veniaDeCostado =
xAnterior + ancho / 2 <=
plataforma.bordeIzquierdo()
-         || xAnterior - ancho / 2 >=
plataforma.bordeDerecho();
-
-         return seSuperponeEnX && seSuperponeEnY
&& veniaDeCostado;
-     }

```

Este método nos permite detectar colisiones laterales y superiores para evitar que las traspase como nos paso en los problemas anteriores.

```

- Disparo Plus:
- if (entorno.sePresionoBoton(entorno.BOTON_DERECHO)
-         && disparosPlus > 0
-         && disparoPlus == null) {
-

```

```

-         disparoPlus = new Proyectil(x, y,
entorno.mouseX() + camaraX, entorno.mouseY(), 28);
-         disparoPlus.hacerAtravesador();
-         disparoPlus.hacerMasLento();
-         disparosPlus = disparosPlus - 1;
-     }

```

Esto nos permite utilizar el disparo Plus, antes de generar el disparo se verifica si el personaje tiene usos disponibles y que no haya otro disparo especial activo en pantalla. Al utilizar este disparo se descuenta una unidad del contador de disparos especiales.

Tuvimos problemas con esto ya que podía utilizarse varias veces seguidas sin esperar que desapareciera, la solución fue agregar la condición `disparoPlus == null`, permitiendo que exista un único disparo especial activo al mismo tiempo.

- Disparo Triple:

```

- public void dispararTriple(double mouseX, double
mouseY) {
-     disparosTriples[0] = new Proyectil(x, y,
mouseX, mouseY - 40, 4);
-     disparosTriples[1] = new Proyectil(x, y,
mouseX, mouseY, 4);
-     disparosTriples[2] = new Proyectil(x, y,
mouseX, mouseY + 40, 4);
- }

- else if
(entorno.sePresionoBoton(entorno.BOTON_DERECHO)
-     && disparosNormalesRealizados >=
10
-     && noHayDisparoTriple()) {
-
-     dispararTriple(entorno.mouseX() +
camaraX, entorno.mouseY());
-     disparosNormalesRealizados = 0;
- }
- }

```

else if (entorno.sePresionoBoton(entorno.BOTON_DERECHO)

```

        && disparosNormalesRealizados >= 10
        && noHayDisparoTriple()) {

            dispararTriple(entorno.mouseX() +
camaraX, entorno.mouseY());
            disparosNormalesRealizados = 0;
        }
    }
}

```

Se implemento también un disparo triple para recompensar al jugador luego de hacer 10 disparos, se realiza esa cantidad de disparos normales y se habilita un disparo triple en simultaneo. Esta mecánica se agrego para ofrecer una mejora al ataque del personaje y tener mayor variedad.

Desde la posición del personaje cada proyectil se dirige hacia la posición cercana del cursor del mouse variando muy poco la coordenada en vertical, de esta manera se logra el disparo triple para mayor capacidad de ataque.

Cuando el jugador ya realizo los diez disparos normales se verifica que no exista otro disparo triple activo para evitar generar varios ataques triples al mismo tiempo. Después de utilizarlo el contador de disparos vuelve a 0.

- Enemigos, generación y solución:

```

private void crearEnemigosSiFaltan() {
    int vivos = 0;

    for (int i = 0; i < enemigos.length; i++) {
        if (enemigos[i] != null) {
            vivos++;
        }
    }

    if (vivos < minimoEnemigos) {
        for (int i = 0; i < enemigos.length; i++)
        {
            if (enemigos[i] == null) {
                enemigos[i] = new
Enemigos(entorno, camaraX);
                break;
            }
        }
    }
}

```



```

    }
}

private void actualizarEnemigos() {
    for (int i = 0; i < enemigos.length; i++) {
        if (enemigos[i] != null) {

            enemigos[i].mover();

            evitarQueEnemigoAtravieseIslas(enemigos[i]);

            if
(enemigos[i].colisionaConPersonaje(personaje)) {
                enemigos[i] = null;
                personaje.perderVida();

                if (personaje.getVidas() <= 0) {
                    juegoTerminado = true;
                }
            }

            if (enemigos[i] != null &&
personaje.getDisparo() != null
                &&
personaje.getDisparo().colisionaConEnemigo(enemigos[i])
) {

                crearItem(enemigos[i].getX(),
enemigos[i].getY());
                enemigos[i] = null;
                personaje.setDisparo(null);
            }

            if (enemigos[i] != null) {
                revisarDisparoTriple(i);
            }

            if (enemigos[i] != null &&
personaje.getDisparoPlus() != null
                &&
personaje.getDisparoPlus().colisionaConEnemigo(enemigos
[i])) {

```

```

        crearItem(enemigos[i].getX(),
enemigos[i].getY());
        enemigos[i] = null;
    }

    if (enemigos[i] != null &&
enemigos[i].seFueDePantalla(entorno, camaraX)) {
        enemigos[i] = null;
    }
}
}

private void
evitarQueEnemigoAtravieseIslas(Enemigos enemigo) {
    for (int i = 0; i <
mapa.getPlataformas().length; i++) {
        if (mapa.getPlataformas()[i] != null
            &&
enemigo.colisionaConPlataforma(mapa.getPlataformas()[i]
)) {

            enemigo.subirUnPoco();
        }
    }
}

```

Este método recorre el arreglo de enemigos para contar cuantas hay vivos. Si la cantidad es menor al establecida, se busca y crea un enemigo nuevo, se mantiene la cantidad en pantalla. Se uso actualizarEnemigos para verificar si toca al personaje, si ocurre colision el enemigo se actualiza como null y la princesa pierde una vida, si las vidas llegan a cero perdiste el juego.

Durante las pruebas que hicimos se nos paso por alto que los enemigos podían atravesar las islas flotantes al desplazarse por el mapa. Para evitar este problema se implemento colisiones entre los enemigos y plataformas, cuando un enemigo hace contacto con la isla se corrige su posición desplazándolo ligeramente hacia arriba con subirUnPoco(). De esta manera se evita que atravesase las plataformas y se mantiene dentro de la pantalla.

CONCLUSION

Durante el desarrollo del proyecto pudimos entender y aplicar mejores conceptos que estuvimos viendo en la materia, objetos, arreglos, organización de códigos. El principal desafío por así decirlo fue implementar colisiones entre personajes, plataformas, enemigos y así como generar el mapa aleatoriamente, respetar cada una de los requerimientos obligatorios y poder mejorar el diseño del programa separando responsabilidades entre clases como escribimos anteriormente como en las clases Mapa y HUD. Creemos que obtuvimos un buen resultado de un juego funcional y esperamos cumplir con las expectativas de un buen proyecto.